MoveMate Host Edition

Backend Developer Guide

Build the engine: models, pipelines, database, Docker and accuracy.

Version	2.0 (API contract v2) - October 2026
Read this if	You build everything in the `backend/` folder, plus Docker and the models.
Start here	Run `docker compose up --build`, open http://localhost:3000, press Load demo data.
Order of reading	Part A and B (shared) -> Part C (video matching) -> Part D (your own guide).

The one rule: the API contract in Part B is the promise between you and your teammate. Part A, B and C are identical in both guides on purpose. If you think the contract is wrong, change it together, in one pull request, never alone.

Part A. The shared picture (both guides contain this part, word for word)

A1. The product in one page

MoveMate Host Edition helps a short-term rental host in four ways. The host scans the home once and records its details, rooms, places and items. Guests get help from a photo. After every stay, a person walks the checklist and the system tells the host whether the home is ready for the next guest. An optional AI video check gives that person hints.

Workflow	Who	Input	Output
A. Baseline scan	Host	Photos of important items, short room-sweep videos	A confirmed inventory: items with name, quantity, reference photo, notes
B. Guest guide	Guest	One photo of an appliance, from a link, no login	Short step-by-step instructions for that exact appliance
C. AI video check (audit)	Cleaner or host	One video per room	Suggestions only: missing, moved, wrong count, could not verify, each with photo evidence
D. Inspection and readiness	Cleaner or host	A checklist per room, photos, counts, notes	Results, cases for anything off, and a readiness report: ready, ready with exceptions, or not ready

Two different checks, on purpose. The AI video check (C) only suggests. The inspection (D) is the human record and the only thing that creates results, cases and readiness. The link between them is one button that copies the AI's suggestions into the checklist as hints.

A2. Words we all use the same way

Word	Meaning
Property, Room, Place	A rental home, the rooms in it, and places inside a room at any depth (Living room > Glass cabinet > Top shelf). A place is stored as a room with a parent.
Item	One kind of thing the host cares about (a Nespresso machine, 4 wine glasses). Has a label, a quantity, a status, a type (kind), a condition and an availability.
Kind	fixed (TV, built in), movable (a remote), shared (bottle opener used in several rooms), consumable (toilet paper: counted, never 'missing').
Condition	new, good, worn, damaged, missing, under_repair. Every change is saved in a history with who, when, note and photo.
Availability	guest or staff_only. Staff-only items (cleaning spray) never appear in the guest guide.
Alternate place	Another place where an item may legitimately be (the TV remote on the TV console). Seeing it there is not 'misplaced'.
High value	A flag on an item. A photo is required before it can be marked OK in an inspection.
Reference crop	The cropped photo of an item taken at scan time. This is what the audit looks for later.
Suggested / Confirmed	The model proposes items (suggested). The host confirms them. Only confirmed items are used by guests and audits.
Baseline	The confirmed inventory made during the scan.
Audit (AI video check)	One AI check of a property from videos. Has findings. Suggestions only.

Word	Meaning
Inspection	One human checklist for a property: after checkout, after cleaning, or before check-in. Has a result per item and cleaning checks per room.
Cleaning status, inventory status	Two separate statuses per room in an inspection. A room can be clean but have a missing item, or complete but dirty.
Case (discrepancy)	A factual record opened when an inspection result is missing, misplaced or damaged. Open, investigating, resolved, with an append-only trail. Not a blame list.
Outcome	How a case ended: found, restored_to_place, repaired, replaced, recorded_missing.
Essential item (blocks readiness)	A case the host marks as blocking. The property cannot be ready while it is open.
Readiness	ready, ready_with_exceptions or not_ready. An inspection being complete does not mean the property is ready.
Exception	The host completes an inspection despite open checks, with a written reason. It never hides an essential gap.
Finding	One result of an audit: missing, misplaced, count_mismatch, unverified (and later extra). It has a resolution: open, found, confirmed or dismissed.
Resolution	open = nobody has looked. found = the cleaner says it turned up. confirmed = the host confirms a real loss. dismissed = the host says it is a false alarm.
Role	cleaner or host. In v1 there is no login, so the link carries the role (see A6).
Unverified / Not seen	Unverified: the video did not show the usual spot. Not seen: the person could not confirm it during an inspection. Neither is 'missing'. Only a person records a loss.
Mock mode	Fake but consistent model answers so the app works with no API keys.

A3. Architecture

```
phone (browser)  --->  Next.js frontend  (http://localhost:3000)
       |                        |
       |  fetch() with NEXT_PUBLIC_API_URL
       v                        v
  FastAPI backend (http://localhost:8000)  --->  Postgres (docker service "db")
       |   |
       |   +--> files in ./data  (served at /media/...)
       +------> model client: mock.py now, meta.py for real models
                (segmentation, labeling, reasoning, audit judging)
```

The browser never calls a model and never holds an API key. All model calls happen in the backend, behind one interface (`backend/app/services/models/base.py`). That is why the frontend can be built completely against mock mode.

A4. How we split the work so nobody blocks anybody

Area	Owner	Other person may...
`frontend/` (everything)	Frontend dev	read only
`backend/` (everything)	Backend dev	read only
`backend/app/schemas.py`, `frontend/lib/types.ts`, `docs/API_CONTRACT.md`	SHARED	change only together, in one PR, both approve

Area	Owner	Other person may...
`docker-compose.yml`, `.env.example`, `.gitignore`	Backend dev	ask first (a broken compose file blocks both of you)
`README.md`, `docs/`	Either	keep short and correct

- **The contract is the promise.** Part B lists every endpoint. If something is not in the contract, it does not exist yet. Ask for it.
- **Never rename or remove a field.** Add new optional fields instead. Both sides must keep working at every step.
- **Branches:** `feat/fe-...` for frontend work, `feat/be-...` for backend work, `contract/...` for shared changes. Merge to `main` at least daily.
- `main` **always runs.** `docker compose up --build` must work on `main` at all times. Test it before you merge.
- **10-minute sync every day.** Three questions: what I shipped, what I need from you, what I am changing in the contract.
- **Blocked? Use mock mode.** The backend already returns believable data (see B4), so the frontend never waits for real models.
- **Found a bug on the other side?** Open an issue with the request you sent, the response you got, and the expected response. Do not patch their folder.

A5. What we build in 10 days, and what comes later

The full product vision is large. For the hackathon we build a **slice that works end to end**, and we write down honestly what is left. Everything marked 'Built' is in the code and contract today. Everything marked 'Later' is not, and the UI must not pretend otherwise.

Area from the product spec	Hackathon slice	Status
Property details, house rules, procedures	Address, type, capacity, details, rules (free text), shown to guests	Built
Rooms and nested places	Rooms with a type; places at any depth; reference photo per room	Built
Items: placement, type, alternates, availability	Kind, usual place, alternate places, guest or staff-only, high value, minimum quantity	Built
Condition and history	Six conditions, each change saved with who, when, note and photo	Built
Visual inventory and printable sheet	Room reference photo, item crops, printable room sheet with tick boxes	Built
Inspections with required checks	Three kinds, per-room checklist, high-value photo, damaged needs note and photo, cleaning checks	Built
Exception approval	Host only, needs a note, never hides essential gaps	Built
Discrepancy cases	Detect, assign (host), search log, evidence, resolve with outcome, append-only trail	Built
Cleaning vs inventory status	Separate status per room	Built

Area from the product spec	Hackathon slice	Status
Readiness report	Ready, ready with exceptions, not ready; printable	Built
Consumables	Counted by quantity, low-stock flag in the report	Slice only. Stock ledger, restock lists and purchasing: Later
AI boundaries	AI only suggests: hints on checklist items, findings need a person	Built (rule enforced in code)
Accounts, roles, permissions	Two roles chosen by link (not secure)	Later: real login, per-property access, cleaner links scoped to one inspection
Bookings and calendar	Reservation reference typed by hand	Later
Notifications	None	Later
Maintenance jobs and costs	A damaged item just opens a case	Later
Dashboards, reports, exports	One readiness report	Later
Offline use, integrations	None	Later

Pitch line: 'We built the loop that matters: know what should be there, check it, record what is wrong, and say clearly whether the home is ready. Everything else is a layer on top.'

A6. Who can confirm what: the cleaner and the host

The system only **suggests** problems. People decide. The cleaner is at the property and can check the drawer. The host owns the final call, because a confirmed loss becomes the evidence record and a false accusation against a cleaner or guest is the worst outcome.

Action on a finding	Cleaner	Host
Run an audit, upload videos	Yes	Yes
Add or edit a note (for example 'it was in the drawer')	Yes	Yes
Mark **found** (the item turned up)	Yes	Yes
Undo their own **found**	Yes	Yes
Confirm loss	No (HTTP 403)	Yes
Dismiss (false alarm)	No (HTTP 403)	Yes
Reopen a finding	Only their own found	Yes

- **Typical flow:** the audit finds a missing remote. The cleaner looks, finds it in a drawer, taps **It was found** and adds a note. The host sees 'Cleaner says it was found, host to review' and taps **Dismiss** or **Confirm loss**.
- **Cleaner cannot find it:** the cleaner leaves it open, the host decides. The host can also confirm directly without the cleaner.

- **No login in v1.** The role is sent with every update (`role: 'cleaner' | 'host'`). The frontend takes it from the link: `/audit/{id}?role=host` for the host (default links from the property page), `/audit/{id}?role=cleaner` for the cleaner (the default when no role is given). The host has a **Copy link for the cleaner** button.
- **This is a stand-in, not security.** Anyone who edits the link can act as host. Before real users, add login, take the role from the signed-in user, and ignore any role sent by the client.
- The server records who resolved each finding (`resolved_by`) and when (`resolved_at`) so the history is clear.

A6b. Inspections, cases and readiness: the rules

These rules are enforced by the server and shown by the screens. Both of you need to know them, so they are here, once.

Action	Cleaner	Host
Start an inspection, set item results, tick cleaning steps, add photos	Yes	Yes
Complete an inspection when all checks are done	Yes	Yes
Complete with an **exception** (needs a written reason)	No (403)	Yes
Add a note, search entry or photo to a case	Yes	Yes
Resolve a case as **found** or **restored_to_place**	Yes	Yes
Resolve as **repaired**, **replaced** or **recorded_missing**	No (403)	Yes
Assign a case, mark it **blocks readiness**	No (403)	Yes

- **Results:** ok, not_seen, misplaced, missing, damaged (and low_stock for consumables, set by the server when the count is under the minimum).
- **Required checks (HTTP 422 if broken):** a high-value item needs a photo before OK. A counted quantity below the expected one cannot be OK. Consumables need a counted quantity and cannot be marked missing.
- **Cases open by themselves** when a result is not_seen, misplaced, missing or damaged. You cannot flip such an item to OK while its case is open (HTTP 409): resolve the case first so the record stays complete.
- **Completing:** blocked (HTTP 409) while checks are pending, with a list of what is left. The host may complete with an exception note. The inspection is then `completed_with_exception`.
- **Readiness:** `ready` = everything done, no open blocking case. `ready_with_exceptions` = completed by exception, no essential gaps. `not_ready` = the inspection is still in progress, or a case marked blocks_readiness is open. An exception never turns a blocking gap into ready.
- **Recorded missing:** a case resolved as recorded_missing sets the item's condition to missing. The next inspection skips it, and the readiness report lists it under known gaps, so nobody is asked to find it again.
- **Not seen is not missing.** Only a person resolves a case as recorded_missing, and only the host.
- **AI hints:** `POST /inspections/{id}/ai-suggestions` copies findings from an AI video check onto the checklist items as `ai_suggestion`. It never changes a result.
- **Wording:** cases are factual ('not found at the usual place'), never accusing. The trail records who did what and when, and entries cannot be edited or deleted.

A7. Running the project

One command starts everything (database, backend, frontend):

```
docker compose up --build
```

What	URL
App	http://localhost:3000
Live API docs (always matches the code)	http://localhost:8000/docs
Health check	http://localhost:8000/api/health

- It works with no `.env` file. To use real models later: `cp .env.example .env`, set `MOCK_MODELS=false`, add keys, run again.
- On the Properties page click **Load demo data (dev)**. You get a ready property with three rooms, nested places, items of every kind and house rules.
- **Upgrading from v1?** The new version adds database columns and tables, and the app does not migrate old data. Run `docker compose down -v` once, then `docker compose up --build`.
- Code changes reload by themselves (backend `--reload`, frontend `next dev`). You only rebuild when dependencies change.
- **Real phone testing:** put the phone on the same Wi-Fi. In `.env` set `NEXT_PUBLIC_API_URL` and `PUBLIC_BASE_URL` to `http://<computer-ip>:8000` and `FRONTEND_ORIGIN` to `http://<computer-ip>:3000`. Restart. Open `http://<computer-ip>:3000` on the phone.
- Reset the database: `docker compose down -v`. Backend logs: `docker compose logs -f backend`.

Part B. The shared API contract (identical in both guides)

Source of truth in code: `backend/app/schemas.py`. **Frontend mirror:** `frontend/lib/types.ts`. **Live version:** http://localhost:8000/docs. A change to any endpoint, field or enum is ONE pull request touching all three of schemas.py, types.ts and docs/API_CONTRACT.md, approved by both people.

B1. Conventions

Topic	Rule
Base URLs	Backend http://localhost:8000. The browser calls it directly using NEXT_PUBLIC_API_URL. Interactive docs at http://localhost:8000/docs are the live version of this contract.
IDs and times	All ids are strings (UUIDs). Timestamps are ISO 8601 strings.
Media URLs	Every *_url field is an absolute URL, ready for an img src. The frontend never builds media URLs.
Boxes	Normalized 0..1 {x, y, w, h}. Multiply by the displayed image size to draw.
Uploads	multipart/form-data. Field is 'file' for one file and 'files' for several. Images max 15 MB, videos max 150 MB.
Errors	Non-2xx responses carry JSON {"detail": "human readable message"}. 400 bad input, 404 not found, 409 audit already running or action not allowed in the current state (inspection completed, case resolved, checks pending), 413 file too large, 422 unreadable video.
Long work	Audits run in the background. POST /run returns 202 at once. Poll GET /audits/{id} every 2 seconds until status is done or failed. The 'stage' text is shown to the user.
Auth	None in v1 (hackathon). Guest access is by the unguessable guest_token. Add host auth after the demo.
Item lifecycle	New scan items are 'suggested'. Only 'confirmed' items are used for guest answers, audits and inspections. Guests never see staff_only items.
Two kinds of check	An AI video check (audit) produces suggestions. An inspection is the human checklist and the only thing that produces results, cases and readiness. The two connect only through POST /inspections/{id}/ai-suggestions, which fills hints.
Cleaning vs inventory	Every room in an inspection has two separate statuses: cleaning_status and inventory_status. Cleaned does not mean complete, and complete does not mean ready.
Consumables	Counted by quantity, never marked missing. Below min_quantity the result becomes low_stock. They are excluded from AI video checks.
Not seen is not missing	'not_seen' means nobody could confirm the item. Only a person (or the host on a case) records 'missing'. The AI never does.
Inspection rules (422 if broken)	High-value items need a photo before OK. A counted quantity below expected cannot be OK. Not seen, misplaced, missing and damaged open a case automatically, and the item cannot be flipped to OK while that case is open (409). Completing is blocked (409) while checks are pending or cases are unresolved, unless the host completes with an exception_note. An exception never hides an essential (blocks_readiness) gap.
Cases	A case is a factual record with an append-only event trail. Statuses: open, investigating, resolved. Cleaners may log searches and notes and resolve only as found or restored_to_place. Only the host may assign, mark blocks_readiness, or resolve as repaired, replaced or recorded_missing (403 otherwise). An item recorded missing is skipped in the next inspection and listed under known_gaps.
Readiness	ready = no blockers and no exception. ready_with_exceptions = host exception, no essential gaps. not_ready = any essential gap, open blocking case, or unfinished room.
Roles on bodies	Every write that has a rule carries role ('cleaner'|'host') and actor (a name). The name is stored for the audit trail. Both are stand-ins until real login exists.

Topic	Rule
Roles (v1, no login)	The client sends role 'cleaner' or 'host' with every finding update. The frontend picks it from the link: /audit/{id}?role=host for the host, ?role=cleaner (the default) for the cleaner. This is a stand-in for real login; it is not secure. Add auth before real users.
Who may resolve a finding	Cleaner: may set 'found' (item turned up) and add a note; may 'open' again only to undo their own 'found'. Cannot confirm or dismiss. Host: may set any resolution, and only the host can confirm a loss. Not allowed -> HTTP 403. Missing role -> HTTP 422. (Applies to AI video check findings. Inspections and cases have their own rules above.)
Finding rules	'unverified' means the camera did not cover the spot, never an accusation. 'missing' is only produced at confidence >= 0.7. 'misplaced' means the item WAS seen but not in its usual place: either elsewhere in its room, or in another room (the observed text says where). 'count_mismatch' means fewer units seen than the saved quantity.

B2. Endpoints

Method	Path	Request	Response	Used in
GET	`/api/health`	-	Health	dev banner
POST	`/api/dev/seed`	-	SeedResult	Properties page button
GET	`/api/properties`	-	PropertyOut[]	Properties list
POST	`/api/properties`	JSON PropertyCreate {name, address?, property_type?, capacity?, details?, rules?}	PropertyOut	Properties list
PATCH	`/api/properties/{id}`	JSON PropertyPatch	PropertyOut	Property page
GET	`/api/properties/{id}`	-	PropertyDetail	Property page
POST	`/api/properties/{id}/rooms`	JSON {name, room_type?, parent_id?}	RoomOut	Property page, Room page
GET	`/api/rooms/{room_id}`	-	RoomOut	Room page
PATCH	`/api/rooms/{room_id}`	JSON {name?, room_type?}	RoomOut	Room page
GET	`/api/rooms/{room_id}/locations`	-	RoomOut[]	Room page
POST	`/api/rooms/{room_id}/photo`	multipart: file (image)	RoomOut	Room page
GET	`/api/rooms/{room_id}/sheet`	-	HTML page	Room page (opens in new tab)
GET	`/api/properties/{id}/rooms`	-	RoomOut[]	Property page
POST	`/api/rooms/{room_id}/baseline/photo`	multipart: file (image), priority (high|normal)	BaselineResult	Room page
POST	`/api/rooms/{room_id}/baseline/video`	multipart: file (video)	BaselineResult	Room page
GET	`/api/rooms/{room_id}/items`	query: status (optional)	ItemOut[]	Room page
PATCH	`/api/items/{id}`	JSON ItemPatch	ItemOut	Room page

Method	Path	Request	Response	Used in
DELETE	`/api/items/{id}`	-	204 no body	Room page
POST	`/api/rooms/{room_id}/items`	JSON ItemCreate	ItemOut	Room page
POST	`/api/items/{id}/condition`	multipart: condition, actor, role, note?, file? (photo)	ItemOut	Room page
GET	`/api/items/{id}/history`	-	ConditionEventOut[]	Room page
POST	`/api/rooms/{room_id}/items/confirm-all`	-	{confirmed: n}	Room page
GET	`/api/guest/{token}`	-	GuestProperty	Guest page
POST	`/api/guest/{token}/ask`	multipart: file (image)	GuestAnswer	Guest page
POST	`/api/properties/{id}/audits`	-	AuditCreated	Property page
POST	`/api/audits/{id}/rooms/{room_id}/video`	multipart: file (video)	UploadAck	Audit page
POST	`/api/audits/{id}/rooms/{room_id}/frames`	multipart: files (images, repeated)	UploadAck	Audit page (optional)
POST	`/api/audits/{id}/run`	-	RunAck (HTTP 202)	Audit page
GET	`/api/audits/{id}`	-	AuditDetail	Audit page (poll every 2 s)
PATCH	`/api/findings/{id}`	JSON {role (required), resolution?, note?}	FindingOut	Audit page
POST	`/api/properties/{id}/inspections`	JSON {kind, inspector, reservation_ref?}	InspectionOut	Property page
GET	`/api/properties/{id}/inspections`	-	InspectionOut[]	Property page
GET	`/api/inspections/{id}`	-	InspectionDetail	Inspection page
PATCH	`/api/inspection-items/{id}`	JSON InspectionItemPatch	InspectionItemOut	Inspection page
POST	`/api/inspection-items/{id}/photo`	multipart: file, actor, role	InspectionItemOut	Inspection page
PATCH	`/api/cleaning-checks/{id}`	JSON {actor, done}	CleaningCheckOut	Inspection page
POST	`/api/inspections/{id}/ai-suggestions`	JSON {audit_id}	{applied: n}	Inspection page
POST	`/api/inspections/{id}/complete`	JSON {role, actor, exception_note?}	InspectionOut	Inspection page
GET	`/api/inspections/{id}/readiness`	-	ReadinessReport	Readiness page
GET	`/api/properties/{id}/discrepancies`	query: status (optional)	DiscrepancyOut[]	Cases page
GET	`/api/discrepancies/{id}`	-	DiscrepancyDetail	Case page
PATCH	`/api/discrepancies/{id}`	JSON {role, actor, status?, assigned_to?, blocks_readiness?}	DiscrepancyOut	Case page

Method	Path	Request	Response	Used in
POST	`/api/discrepancies/{id}/events`	multipart: role, actor, kind, text?, file?	DiscrepancyEventOut	Case page
POST	`/api/discrepancies/{id}/resolve`	JSON {role, actor, outcome, note?}	DiscrepancyOut	Case page
GET	`/media/{path}`	-	image bytes	all pages (img src)

B3. Data shapes

Name	Shape
`BBox`	{ x, y, w, h } numbers 0..1, relative to the image width and height (top-left origin)
`RoomOut`	{ id, property_id, parent_id|null, name, room_type, path ('Living room > Closet'), reference_photo_url|null, confirmed_items, suggested_items }. A place inside a room has parent_id set
`PropertyOut`	{ id, name, guest_token, created_at, address|null, property_type|null, capacity|null, details|null, rules|null }
`PropertyDetail`	PropertyOut + { rooms: RoomOut[] (top-level only), latest_audit_id|null, latest_inspection_id|null, open_discrepancies }
`ItemOut`	{ id, room_id, label, category|null, quantity, priority, bbox|null, crop_url|null, source_image_url|null, location_hint|null, notes|null, status 'suggested'|'confirmed'|'retired', location_id|null, location_path|null, kind 'fixed'|'movable'|'shared'|'consumable', condition 'new'|'good'|'worn'|'damaged'|'missing'|'under_repair', availability 'guest'|'staff_only', high_value, min_quantity|null, alternate_locations: string[] }
`ItemCreate`	{ label, quantity? (1..999), kind?, category?, location_id?, location_hint?, notes?, availability?, high_value?, min_quantity?, alternate_locations? }
`ItemPatch`	any of { label, quantity (1..999), notes, location_hint, status, kind, availability, high_value, min_quantity, alternate_locations, location_id (null = anywhere in the room) }
`ConditionEventOut`	{ id, item_id, condition, previous|null, note|null, photo_url|null, actor, role|null, created_at }
`BaselineResult`	{ capture_id, items: ItemOut[] } (new items arrive with status 'suggested')
`GuestProperty`	{ property_name, item_labels: string[], house_rules|null } (staff_only items are never listed)
`GuestAnswer`	{ matched, message, item_label|null, guide: GuestGuide|null }
`GuestGuide`	{ title, steps: string[], warnings: string[], seen, unsure: string[] }
`AuditCreated`	{ id, property_id, status }
`UploadAck`	{ audit_id, room_id, frames }
`RunAck`	{ audit_id, status }
`FindingOut`	{ id, audit_id, item_id|null, room_id|null, item_label, type 'missing'|'misplaced'|'count_mismatch'|'extra'|'unverified', expected|null, observed|null, confidence|null, evidence_frame_url|null, reference_crop_url|null, resolution 'open'|'found'|'confirmed'|'dismissed', resolved_by 'cleaner'|'host'|null, resolved_at|null, note|null }
`AuditDetail`	{ id, property_id, status 'draft'|'running'|'done'|'failed', stage|null, summary|null, rooms_uploaded: room_id[], findings: FindingOut[], created_at }
`SeedResult`	{ property_id, guest_token, room_ids[] }
`Health`	{ status, mock_models }

Name	Shape
`InspectionOut`	{ id, property_id, kind 'after_checkout'|'after_cleaning'|'before_checkin', inspector, reservation_ref|null, status 'in_progress'|'completed'|'completed_with_exception', exception_note|null, exception_by|null, created_at, completed_at|null }
`AiSuggestion`	{ type (a finding type), observed|null, confidence|null, evidence_frame_url|null, audit_id } (a hint, never a result)
`InspectionItemOut`	{ id, inspection_id, room_id, item_id|null, label, kind, high_value, expected_quantity, min_quantity|null, result 'pending'|'ok'|'not_seen'|'misplaced'|'missing'|'damaged'|'low_stock', observed_quantity|null, note|null, photo_url|null, checked_by|null, checked_at|null, reference_crop_url|null, location_path|null, alternate_locations[], ai_suggestion|null, discrepancy_id|null }
`CleaningCheckOut`	{ id, room_id, label, done, done_by|null, done_at|null }
`RoomInspection`	{ room_id, room_name, room_type, cleaning_status, inventory_status ('pending'|'in_progress'|'done'|'not_applicable'), items: InspectionItemOut[], cleaning: CleaningCheckOut[] }
`InspectionDetail`	InspectionOut + { rooms: RoomInspection[], blockers: string[], can_complete }
`InspectionItemPatch`	{ role, actor, result?, observed_quantity?, note? }
`ReadinessReport`	{ inspection_id, property_name, kind, inspector, reservation_ref|null, status 'ready'|'ready_with_exceptions'|'not_ready', cleaning_status, inventory_status, summary, rooms: [{room_name, cleaning_status, inventory_status, issues[]}], blocking[], open_cases[], known_gaps[], low_stock[], exception_note|null, exception_by|null, generated_at }
`DiscrepancyOut`	{ id, property_id, inspection_id|null, item_id|null, room_id|null, room_name|null, item_label, type 'missing'|'misplaced'|'damaged'|'not_seen', status 'open'|'investigating'|'resolved', assigned_to|null, blocks_readiness, outcome 'found'|'restored_to_place'|'repaired'|'replaced'|'recorded_missing'|null, resolution_note|null, resolved_by|null, resolved_at|null, created_at }
`DiscrepancyEventOut`	{ id, kind 'detected'|'note'|'search'|'evidence'|'assigned'|'status'|'resolved', text|null, photo_url|null, actor, role|null, created_at }
`DiscrepancyDetail`	DiscrepancyOut + { events: DiscrepancyEventOut[], reference_crop_url|null, expected_location|null, ai_suggestion|null }

B4. Mock mode: what the frontend can rely on

- MOCK_MODELS=true is the default. No keys needed. Answers are fake but consistent, so every screen can be built.
- POST /api/dev/seed creates 'Demo loft (seeded)' with a Living room, a Kitchen and a Bathroom, nested places (Living room > Glass cabinet > Top shelf, Kitchen > Upper cupboard > Left shelf, Under-sink cupboard), a mix of item kinds (movable, fixed, consumable), high-value items, items with an alternate place, and one suggested item to review. It also stores house rules.
- Seeded AI video check behaviours (5 findings): 'Black Roku remote' is missing (searched in the other room too, still missing), 'Samsung TV remote' is misplaced (seen on the sofa, usually on the coffee table), 'Bottle opener' is misplaced (kitchen item found in the Living room), 'Wine glass set' is count_mismatch (3 of 4), 'Smart speaker' is unverified. The rest pass. The TV remote's alternate place is 'TV console', not 'sofa', so it stays misplaced.

- Any item whose notes contain [mock:missing], [mock:unverified], [mock:count=N], [mock:moved=some place] or [mock:otherroom] triggers that outcome in an AI video check, so the frontend can create its own test cases by editing notes. ([mock:otherroom] needs a second room with an uploaded video in the same audit.)
- Inspections need no model: they are human checklists, so the whole inspection, case and readiness flow works the same with or without mock mode.
- Any video works in mock mode, as long as it is a real video file (ffmpeg extracts frames from it).

B5. Example calls (copy and run)

```
# create a property and a room
curl -s -X POST localhost:8000/api/properties -H 'Content-Type: application/json' -d '{"name":"Beach flat"}'
curl -s -X POST localhost:8000/api/properties/<PID>/rooms -H 'Content-Type: application/json' -d '{"name":"Kitchen

# baseline photo -> suggested items
curl -s -F file=@kitchen.jpg localhost:8000/api/rooms/<ROOM_ID>/baseline/photo

# confirm everything the model suggested
curl -s -X POST localhost:8000/api/rooms/<ROOM_ID>/items/confirm-all

# audit: create, upload a video for a room, run, poll
curl -s -X POST localhost:8000/api/properties/<PID>/audits
curl -s -F file=@walk.mp4 localhost:8000/api/audits/<AID>/rooms/<ROOM_ID>/video
curl -s -X POST localhost:8000/api/audits/<AID>/run
curl -s localhost:8000/api/audits/<AID>

# inspection: start, set one result, complete, read the report
curl -s -X POST localhost:8000/api/properties/<PID>/inspections -H 'Content-Type: application/json' -d '{"kind":"a
curl -s localhost:8000/api/inspections/<IID>
curl -s -X PATCH localhost:8000/api/inspection-items/<IIID> -H 'Content-Type: application/json' -d '{"role":"clean
curl -s -X POST localhost:8000/api/inspections/<IID>/complete -H 'Content-Type: application/json' -d '{"role":"hos
curl -s localhost:8000/api/inspections/<IID>/readiness
```

Part C. How a video confirms the photos you took earlier

Remember: everything in this part produces **suggestions**. A finding from the AI video check never becomes a result, a case or a readiness decision until a person acts on it (A6 and A6b). **The question:** the host photographs certain items at scan time. Later a video is recorded. Can the system tell, from the video, whether each photographed item is still there? **Yes. This is exactly what the audit does**, in six steps.

Step	What happens	Where
1. Save references	At scan time every confirmed item keeps its **reference crop** (close photo of the item), the **source photo** (wide shot showing where it sits), its label, quantity, location hint and host notes.	Baseline pipeline, tables `items`
2. Keyframes	The room video is cut into still frames, 2 per second, capped at 40 per room.	`services/frames.py` (ffmpeg)
3. Shortlist frames	For each item, pick the 4 frames most likely to show it, by labeling objects in a sample of frames and ranking frames whose labels resemble the item (the match is by what the item looks like, not where it is). If nothing matches, use 4 evenly spread frames.	`pipeline/matching.py`, `pipeline/verdicts.py`
4. Judge	Send the reference crop, the wide source photo and the frames to the vision model. It answers **present**, **not_visible** or **missing**, plus a count, the best evidence frame, a confidence, **where it saw the item and whether that is its usual spot**, and one sentence of reasoning.	`client.judge(...)` in `services/models`
5. Second look and other rooms	Not clearly present? Look again with 8 frames; `missing` only survives if both passes agree. Still not present? Search the frames of the other rooms in the same audit (up to 3 rooms).	`check_item()` in `pipeline/verdicts.py`
6. Turn the answer into a finding	Seen elsewhere (this room or another) -> `misplaced`. Missing at confidence 0.7 or more -> `missing`. Fewer seen than the saved quantity -> `count_mismatch`. Not visible or low confidence -> `unverified`. Present in its place -> no finding. A person resolves every finding (A6).	`check_item()` in `pipeline/verdicts.py`

What if an item was moved?

The system recognises an item by **how it looks** (the reference crop), not by where it sits, so a moved item can still be found if the video shows it. What happens depends on where it ended up:

Where the item is now	Result	Example text on the finding
Elsewhere in the same room, filmed	`misplaced`	Seen on the sofa, left cushion. Usually: On the coffee table.
In another room that was filmed in the same audit	`misplaced`	Found in Living room, on a surface. It belongs in Kitchen.
Anywhere that was not filmed (drawer, bag, off-camera)	`missing` or `unverified`	Not seen in the walkthrough. (Unverified if the area was not covered.)
Still in its usual spot	no finding	-

- **To catch a move into another room, film that room in the same audit.** The cross-room search only uses frames uploaded to the current audit, and checks at most 3 other rooms.
- **Alternate places:** if the host lists another acceptable place for an item (the remote on the TV console), seeing it there is **not** reported as misplaced.

- **Consumables are excluded** from the AI video check. They are counted by a person in an inspection.
- A `misplaced` finding is not a loss. The cleaner can tap **I put it back**, the host can confirm that it moved or dismiss it.
- Moved items are the hardest case for the model: it must be sure it is the same object, not a similar one. Confidence below 0.6 is ignored, which means a real move can occasionally go unreported. The test set (backend guide D7) measures this.

What the user sees

Each finding card shows the **baseline photo** next to the **evidence frame** from the video, so the person can judge in one second. Model confidence is shown as a secondary detail, never as the proof.

What this can and cannot do (be honest in the demo)

Works well	Weak spots, and how the design handles them
Distinct, reasonably large items: TV remote, coffee machine, lamp, router, a set of glasses in plain view.	**Identical-looking items** (two white remotes): the model may call the wrong one present. Mitigation: location hints, host notes, human confirmation.
Items in the same spot as in the scan.	**Hidden items** (inside a drawer, behind a cushion): not visible, so the answer must be `unverified`, never `missing`. The cleaner opens the drawer on camera.
Well-lit, slow, steady video that pauses on each spot.	**Blur, glare, dark rooms, fast panning.** Mitigation: capture tips in the UI, `unverified` when unsure, re-film prompt.
Counting 2 to 6 clearly separated objects.	**Large counts or stacked items** (towels): counts are unreliable. Host sets the quantity; flag only when the shortfall is clear.

> **Accuracy is measured, not assumed.** The mock mode only proves the plumbing. Real accuracy depends on the real models, which are not tested yet. The backend dev must run the test set in section D7 of the backend guide and report real numbers before the demo.

Capture rules that make matching work (frontend shows these as tips, host and cleaner follow them)

- **Reference photo:** one close photo per important item, item filling at least a third of the frame, plus one wide photo showing where it normally sits.
- **Open drawers and cabinets fully** and hold still for one second, in the scan and in the audit.
- **Audit video:** walk the same route each time, 20 to 30 seconds per room, pause one second at every shelf, table and counter, good light, no flash.
- **Re-scan** after the host rearranges the room or buys new items.

Part D. Backend-only guide: you build the engine

D1. Your mission

Own `backend/`, Docker, the database and the models. You make the model workflows **actually work with real models**, keep the API exactly as the contract says (Part B), keep the inspection and case rules correct (A6b), and make the AI check trustworthy. The skeleton you received already runs end to end in mock mode, so your work is replacing fakes with real calls, one at a time, without breaking the frontend.

In scope	Out of scope
Model integration, pipelines, accuracy tuning, tests, Docker, retention and privacy basics, cost limits.	Anything in `frontend/`. Login and accounts, bookings, notifications, stock ledger, maintenance, dashboards (see A5). Training models. Native apps.

D2. Stack and folder map

Tool	Why
Python 3.11, FastAPI, Uvicorn	Fast to write, automatic /docs that doubles as the live contract.
SQLAlchemy 2 + Postgres 16 (docker service db)	Tables created at startup (`create_all`). Tests use SQLite.
Pillow, ffmpeg (installed in the backend image)	Crop images, cut videos into frames.
httpx	Calls to Meta and Baseten/Modal.
pytest	Tests. Run: `docker compose run --rm backend pytest`

```
backend/
  Dockerfile  requirements.txt
  app/
    main.py                creates app, CORS, /media, mounts routers under /api
    config.py              all settings from environment variables
    db.py  models.py       engine + tables
    schemas.py             THE CONTRACT (shared file, change with a contract PR)
    routers/
      properties.py items.py inspections.py discrepancies.py
      baseline.py guest.py audits.py dev.py common.py
    pipeline/
      inspection_logic.py  PURE rules: result validation, blockers, readiness (no database)
      permissions.py       who may do what (findings, cases, exceptions)
      verdicts.py          PURE: judge answers -> finding type (testable with the mock client)
      baseline.py          photo/video -> suggested items
      guest.py             guest photo -> matched item -> cached guide
      audit.py             video frames -> findings (the reference-matching funnel)
      matching.py          iou, dedupe, label similarity, frame shortlist
    services/
      sheet.py             printable room sheet (HTML)
      locations.py         place paths ("Living room > Glass cabinet > Top shelf")
      storage.py           save/read files under DATA_DIR, url_for()
      imaging.py           rotate, resize, crop by normalized box
      frames.py            ffmpeg keyframes
      models/
        base.py            the interface every model client implements
        mock.py            fake answers (default)
        meta.py            REAL calls, full of TODO(backend) markers
  tests/test_flow.py       full AI flow, and the property + inspection + case flow, in mock mode
```

D3. Local development

- `docker compose up --build` runs everything. The backend reloads when you save a file (the folder is mounted).
- Interactive API: http://localhost:8000/docs. Use it to try endpoints.
- Database shell: `docker compose exec db psql -U movemate movemate`. Reset everything: `docker compose down -v`. **Do this after every pull that changes** `models.py`: tables are created at startup with `create_all`, which adds missing tables but never adds columns to an existing one. Before real users, replace it with Alembic migrations.
- Files you create land in `./data` (images, crops, frames), served at `/media/...`.
- Switch to real models by setting `MOCK_MODELS=false` in `.env` and restarting the backend. You can run real and mock side by side by keeping two `.env` files.
- Tests: `docker compose run --rm backend pytest -q`. They run the whole flow including a real ffmpeg video. Keep them green before every merge.
- Add a dependency: put it in `requirements.txt` and run `docker compose build backend`.

D4. Data model

Table	Key columns	Notes
properties	id, name, guest_token, address, property_type, capacity, details, rules	guest_token is the unguessable key in the guest link.
rooms	id, property_id, parent_id, name, room_type, reference_photo_path	A place inside a room is a row with `parent_id` (any depth). `parent_id` is a plain string column, not a foreign key, to avoid join ambiguity.
captures	id, room_id, audit_id, kind, media_path, frame_paths[]	One row per upload. kind: baseline_photo, baseline_video, audit_video, audit_frames, guest_photo.
items	id, room_id, label, category, quantity, priority, bbox, crop_path, source_image_path, location_hint, notes, instructions, status, location_id, kind, condition, availability, high_value, min_quantity, alternate_locations[]	bbox is normalized 0..1. instructions caches the guest guide, cleared when label or notes change. `location_id` is a plain string to a place.
condition_events	id, item_id, condition, previous, note, photo_path, actor, role, created_at	Append-only history of condition changes.
audits, findings	as in v1	AI video check. type: missing, misplaced, count_mismatch, unverified (extra reserved). resolution: open, found, confirmed, dismissed.
inspections	id, property_id, kind, inspector, reservation_ref, status, exception_note, exception_by, completed_at	status: in_progress, completed, completed_with_exception.
inspection_items	id, inspection_id, room_id, item_id, label, kind, high_value, expected_quantity, min_quantity, result, observed_quantity, note, photo_path, checked_by, checked_at, ai_suggestion (json)	A snapshot of each confirmed item at inspection time, so later edits do not rewrite history.
cleaning_checks	id, inspection_id, room_id, label, done, done_by, done_at	Defaults per room type. After-checkout inspections have none.

Table	Key columns	Notes
discrepancies	id, property_id, inspection_id, inspection_item_id, item_id, room_id, item_label, type, status, assigned_to, blocks_readiness, outcome, resolution_note, resolved_by, resolved_at	type: missing, misplaced, damaged, not_seen. status: open, investigating, resolved.
discrepancy_events	id, discrepancy_id, kind, text, photo_path, actor, role, created_at	Append-only trail. Never update or delete rows here.

Ids are string UUIDs. Paths in the database are **relative** to `DATA_DIR`. Convert to URLs only at the edge with `storage.url_for()` (that is the only place that knows the public base URL).

D5. The model layer (do this first, on day 1)

Pipelines call only the interface in `services/models/base.py`. That is the whole trick: you can change vendors, add caching or add retries in one folder.

Method	Job	Real implementation
`segment(image)`	Find object regions. Return normalized boxes.	SAM 3.1 via the Meta API
`label(crop)`	Name one cropped object (brand and model if visible).	Muse Image
`choose_candidate(image, candidates)`	Pick the right item from a short list (guest matching).	Muse Spark (or the same model as judge)
`guide(image, label, notes)`	Write guest steps, grounded in host notes.	Muse Spark
`judge(reference_crop, context_image, frames, label, quantity, location_hint, notes)`	**Reference matching.** Is the item from the baseline photo visible in these video frames?	Open-weights vision model on Baseten/Modal, or Muse Spark if it takes several images

Day-1 verification checklist (before writing any real call)

The model names and endpoints in the brief came from the hackathon materials and have **not** been verified. Open the hackathon docs and answer these for every model, then write the answers in `docs/MODELS.md`:

- [ ] Does it take an image as base64, as a URL, or both? (The skeleton sends base64.)
- [ ] Does it accept several images in one request? How many? (Needed for `judge`.)
- [ ] Does it accept video directly? Maximum length and size? (The skeleton does NOT depend on this: it cuts frames itself.)
- [ ] Does SAM 3.1 return boxes, masks or both? In pixels or fractions? (You must output normalized 0..1 boxes.)
- [ ] Does the segmentation API offer video tracking with persistent object ids? (If yes, use it later to dedupe; the skeleton works without.)
- [ ] Rate limits and the credit budget. Estimate the cost of one baseline photo and one audit room.
- [ ] Can you force JSON output? If not, keep the defensive JSON parser in `meta.py`.
- [ ] Save one working request and response for each model in `MODELS.md`.

Then implement `meta.py` in this order, running `pytest` after each step with `MOCK_MODELS=true` to prove you did not break the contract: **1.** `_post` and `_chat` helpers, **2.** `segment`, **3.** `label`, **4.** `guide`, **5.** `choose_candidate`, **6.** `judge`. Search the file for `TODO(backend)`: every guess you must confirm is marked.

Prompts that are already written (in `meta.py`)

- **Label prompt:** returns JSON `{label, category, confidence}`. Ask for brand and model when visible.
- **Guide prompt:** says to trust host notes over assumptions and to say what is not visible instead of guessing. Keep that rule.
- **Judge prompt:** the most important one. It forces three honest states (present, not_visible, missing), forbids saying `missing` unless the usual spot is clearly visible, and asks for a count and an evidence frame. Do not weaken it.
- Always call judging with temperature 0 and validate the JSON. Retry once on bad JSON, then raise (the audit marks itself failed with a readable stage message).

D5b. The inspection and case logic (no models involved)

This is plain business logic. It must be boringly correct, because it is the record the host relies on. The rules are in A6b. Where they live:

Rule	Where	Test it with
Result validation (high-value photo, count below expected, consumable counts, low_stock)	`pipeline/inspection_logic.py: validate_result`	Unit tests, no database
Blockers and `can_complete`	`compute_blockers`	Unit tests
Readiness status	`readiness_status`, `case_blocks_readiness`	Unit tests: a blocking case stays blocking until resolved with found, restored, repaired or replaced
Who may do what	`pipeline/permissions.py`	Unit tests plus 403 checks in `test_flow.py`
Case opens, reopens or closes when a result changes	`routers/inspections.py: _sync_case`	`test_property_inspection_and_cases`
Outcome changes the item's condition (recorded_missing sets missing, repaired sets good, replaced sets new, found clears missing)	`routers/discrepancies.py`	Same test
Next inspection skips items recorded missing; readiness lists them as known gaps	`create_inspection`, `readiness`	Same test
AI hints import (hints only, never results)	`POST /inspections/{id}/ai-suggestions`	Check the result stays `pending` after import

- Keep `inspection_logic.py` free of database and FastAPI imports. That is what lets it be tested in milliseconds and reused when login and notifications arrive.
- Every state change on a case appends a `discrepancy_events` row with actor and role. No endpoint edits or deletes events.
- Roles still come from the request body (A6). Check them in one place (`permissions.py`) so swapping in real login later changes one file.

D6. The pipelines in detail

Baseline photo (`pipeline/baseline.py: process_photo`)

- Fix phone rotation and shrink to max 1600 px (`imaging.normalize_upload`). Save as the source image.
- `segment` -> drop tiny boxes (area under 0.2% of the image) -> keep the best 20 (`MAX_SEGMENTS_PER_IMAGE`).
- For each box: crop (with 2% padding), save, `label` the crop, create an `Item` with status `suggested`.
- The frontend shows them on a review screen. Nothing the model finds is trusted until the host confirms.

Baseline video sweep (`process_video`)

- ffmpeg -> frames at 2 fps (max 40). Process every 3rd frame, at most 10 frames, to control cost.
- Dedupe objects across frames: same label and box overlap (IoU over 0.3) = same object; keep the highest-confidence crop (`matching.dedupe`).
- Items get `priority = normal`. If SAM video tracking is available, group by track id instead of IoU.

Guest guide (`pipeline/guest.py`)

- Segment the guest photo, take the largest object, label it, rank the property's confirmed items by label similarity, take the top 5.
- `choose_candidate` picks among those 5 (far more reliable than free-form recognition). Below confidence 0.6, answer 'no match' honestly.
- If the item already has cached `instructions`, return them instantly. Otherwise `guide`, store, return.
- Rate-limit this endpoint (see D9): it is public.

AI video check (`pipeline/audit.py`) - see Part C for the six-step story. Suggestions only

```
POST /audits/{id}/run  ->  202 immediately, work continues in a background task
rooms = frames of every room uploaded to this audit (labels computed lazily, only when needed)
for each confirmed item (pipeline/verdicts.py: check_item):
    pass 1: judge with the 4 best frames of its room          (scope "room")
    not present? pass 2 with 8 frames; 'missing' needs both passes to agree
    still not present? search up to 3 OTHER rooms (4 frames each)   (scope "other_rooms")
        found there  -> misplaced  ("Found in Living room ... It belongs in Kitchen.")
    present but at_usual_spot is false -> misplaced ("Seen on the sofa. Usually: coffee table.")
    present, fewer than expected      -> count_mismatch
    missing at >= 0.7 confidence      -> missing, else unverified
    not visible                        -> unverified
skip consumables (counted by a person) and items whose alternate place matches where the item was seen
audit.stage updated before every item and every cross-room search (the UI shows it)
audit.summary = "N items checked: X issues, Y could not be verified, Z OK."
on any exception: status = failed, stage = "Failed: <short reason>"
```

- **Never produce `missing` from a view that did not cover the item's spot.** That is `unverified`.
- `MIN_MISSING_CONFIDENCE` (0.7) turns doubtful `missing` into `unverified`. `MIN_MISPLACED_CONFIDENCE` (0.6) ignores doubtful 'seen elsewhere' answers, which avoids false moves but can hide a real one.
- The decision logic is in `pipeline/verdicts.py` and has no database code, so you can unit test it with the mock client (see `RoomFrames` and `check_item`). Keep it that way.
- `judge` must return `where` and `at_usual_spot`. The real prompt in `meta.py` asks for both. Confirm the real model fills them in; if it returns null, moves inside a room will be missed (cross-room finds still work).

- The evidence frame is the frame the model named, mapped back through `picks` to the stored frame path.
- Add the planned fallback flag `AUDIT_MODE=direct`: skip the frame-labeling step (`_observe`) and send evenly spread frames. It is cheaper, has fewer moving parts, and is your safety net if segmentation misbehaves on demo day.

D7. Making the video-matching accurate, and proving it

The skeleton gets the plumbing right. Whether the real models recognise **the specific item from the photo** inside a moving video is something you must measure, then tune.

Tuning levers, cheapest first

Lever	How
Better reference	Send BOTH the close crop and the wide source photo to `judge` (the skeleton passes `context_image`; make sure `meta.py` actually sends it as an extra image and the prompt mentions it).
Better frames	Increase shortlist size from 4 to 6 for small items; sample 3 fps; drop blurry frames (variance of Laplacian under a threshold) before judging.
Location hints	Fill `location_hint` automatically at baseline ('left of the lamp, on the coffee table') using one extra model call, and include it in the judge prompt.
Two-pass judging	**Already built** (`verdicts.py`): not present -> second look with 8 frames, `missing` needs both passes. Tune the frame counts (`FIRST_PASS`, `SECOND_PASS`).
Host input	Make notes and quantity easy to set (frontend). Counts are the weakest model skill: host-confirmed counts help most.
Embeddings (optional, heavy)	A local image-embedding model (for example CLIP) can pre-rank frames by similarity to the reference crop. It adds torch to the image, so only try it if the cheap levers fail.

The test set (build it by day 8)

- Pick your demo property: 2 rooms, about 15 confirmed items, scanned with real photos.
- Record 15 audit videos: 3 with everything in place, 4 with one known item removed, 3 with an item moved inside its room, 2 with an item moved to another room (film both rooms), 2 with a wrong count, 1 where a spot is deliberately not filmed.
- Run each through the audit and fill this table. Report these real numbers in the pitch, never hoped-for ones.

Case	Expected	Got	Correct?	Notes (blur, glare, wrong frame...)
Everything present (x3)	No findings			
Remote removed (x4)	missing			
Remote moved to the sofa (x3)	misplaced			
Kitchen item moved to the living room (x2)	misplaced			
3 of 4 glasses	count_mismatch			
Spot not filmed	unverified			

Track: **removed items caught**, **moves caught** (inside the room and across rooms), **false alarms** (flagged but present), **unverified rate**. A system with some `unverified` and zero false accusations is better than one that never says `unverified` and sometimes accuses wrongly.

D8. Endpoint checklist (what the skeleton gives you, what you finish)

Endpoint group	Skeleton status	Your job
Properties, rooms, places, items, condition	Complete	Add validation edge cases and tests. Keep the contract.
Inspections, cleaning checks, readiness	Complete (logic checked in isolation, full flow test written)	Run `test_property_inspection_and_cases` first: it was written without a running database. Fix whatever it finds.
Cases (discrepancies)	Complete	Same. Add a test for each 403 in A6b.
Printable room sheet	Complete	Check it prints on one page for the demo room.
Baseline photo and video	Works with mock	Real `segment` and `label`; tune caps; log cost.
Guest ask	Works with mock	Real `choose_candidate` and `guide`; add rate limiting.
Audit upload, run, poll	Works with mock	Real `judge`; shortlist; thresholds; `AUDIT_MODE=direct`.
Finding roles (`PATCH /findings/{id}`)	Complete	Rules live in `pipeline/permissions.py` (`can_set`). Keep the table in A6 and the code in step. When login arrives, derive the role from the user, not the request body.
Dev seed	Complete	Keep behaviours stable: the frontend depends on them (B4).
Not built yet	-	Auth, migrations, retention job, usage table, rate limiting (D9), and everything marked Later in A5.

D9. Reliability, cost, security, privacy (do these by day 9)

Topic	What to do
Cost caps	Already in config: 40 frames per room, 20 segments per image, 10 frames per baseline video. Add a `usage` table (route, model, units, estimated cost) and log every model call so you can quote a cost per audit.
Caching	Cache label results by crop hash. Guest guides are cached on the item. Never re-judge an unchanged room.
Failure handling	Wrap model calls: retry once, then raise. `run_audit` already catches everything and sets `failed` with a short message. Test by pointing `META_API_BASE` at a wrong URL.
Rate limiting	Public guest endpoint: limit per IP (for example 10 per hour) with a small in-memory or table-based counter; return HTTP 429 with a friendly `detail`. Tell the frontend dev when it ships.
Upload safety	Size caps exist (15 MB image, 150 MB video). Reject non-image files by trying to open with Pillow (already done in `imaging`).
Secrets	Keys only in `.env`, never in code or logs. `.env` is git-ignored. Do not log request bodies or image bytes.
Privacy	Guest photos: delete after 24 h. Audit videos and frames: delete after 30 days except evidence frames attached to findings. Write a daily cleanup task; delete files and rows together.
Migrations	`create_all` cannot alter existing tables. For the demo, `docker compose down -v` after a model change. Before real users, add Alembic.

Topic	What to do
Auth (after the demo)	Host endpoints are open in v1 and the finding, inspection and case roles come from the request body, so they are **not secure**. Before real users, add login, take the role from the signed-in user, check property ownership on every route, and scope cleaner links to one audit.
Storage later	To move to Supabase or S3, keep the three function signatures in `services/storage.py` and change only their bodies.

D10. Your 10-day plan

Days	Goal	Deliverables	Gate to pass
1-2	Foundation	**First run of everything**: `docker compose up --build`, then `pytest`. The v2 code has never been executed, so expect to fix small things. Verify the model docs, fill `MODELS.md`. Real `segment` + `label` working in a small script.	Stack up, both tests green, one real photo goes through segment + label.
3-4	Baseline scan	Real baseline photo and video pipelines, caps, dedupe tuned, cost logged.	Demo kitchen fully scanned and sensible.
5	Guest guide	Real `choose_candidate` and `guide`, caching, rate limit, host notes flow through to guides, staff-only items hidden.	10 test photos return the right appliance guide.
6	Inspection and cases	Harden the rules in A6b with tests for every 403, 409 and 422. Make sure the demo data lets you reach all three readiness outcomes by hand.	Ready, ready with exceptions and not ready each reachable in the demo (frontend guide D9).
7-8	AI video check	Real `judge`, frame shortlist, thresholds, fallback flag, evidence frames, build the 15-video test set.	Removed items flagged, unverified when unsure.
9-10	Polish and demo	Measure and report real accuracy, retention task, failure tests, demo data frozen, rehearsals with the frontend dev.	Three clean rehearsals plus a backup recording.

D11. Testing

- `pytest` has two tests: the full AI flow, and `test_property_inspection_and_cases` (property details, nested places, condition history, inspection, high-value photo rule, case open and resolve, exception, readiness, recorded-missing skip). The pure modules (`inspection_logic.py`, `verdicts.py`, `permissions.py`) need no database, so add direct unit tests for them as you go. `pytest` covers the full mock flow: seed, baseline, confirm, video upload, audit run, findings, guest ask, and the cleaner/host rules (cleaner gets 403 on confirm and dismiss, can mark found, host confirms, cleaner cannot undo a host decision, missing role gives 422). Add a test whenever you change behaviour.
- Add **contract tests**: after any change to `schemas.py`, check `http://localhost:8000/openapi.json` still contains every endpoint in Part B.
- Keep a `scripts/try_models.py` (you write it) that calls each real model once with a sample image and prints the raw response. It saves hours when a vendor changes something.
- Before each merge: `docker compose down -v && docker compose up --build`, load demo data, click through all three workflows once.

D12. Troubleshooting

Symptom	Likely cause and fix
Backend exits at start with a database error	Postgres not ready or wrong URL. `docker compose logs db`. Compose waits for the health check; if you changed credentials, run `docker compose down -v`.
`ffmpeg failed` (HTTP 422)	The upload is not a readable video. Test the file locally with `ffmpeg -i file`. Some browsers upload `.mov` HEVC; the image's ffmpeg handles it, check the stderr text.
Frontend gets CORS errors	`FRONTEND_ORIGIN` must equal the exact origin in the browser, including IP and port.
Images 404 or point to localhost on a phone	`PUBLIC_BASE_URL` must be the computer's LAN IP when testing on a phone.
Audit stuck on 'running'	The background task died. `docker compose logs backend`. The audit sets `failed` on exceptions; a hard crash leaves `running`, so restart and re-run (run is allowed again only after setting status back to draft: add a small admin reset if needed).
Real model returns prose around the JSON	The parser extracts the first JSON object. If it still fails, tighten the prompt and add a retry with 'Return JSON only'.
Costs rising fast	Check caps, enable `AUDIT_MODE=direct`, cache labels, and lower frames per room.